ESHRAGH UNIVERSITY

Department of Computer Engineering — Embedded Systems



Longest Remaining Time First (LRTF) CPU Scheduling Algorithm: Mathematical Analysis, Implementation, Synchronization, and Comparative Evaluation

A Research Paper in Operating Systems and Embedded Real-Time Computing

STUDENT	Zoha Karimifar
INSTRUCTOR	Majid Shamshirgaran
COURSE	Embedding Systems
DEPARTMENT	Computer Engineering
UNIVERSITY	Eshragh University

Submission Date: 21 October 2026

Abstract

CPU scheduling is a fundamental operating-system function that determines which ready process executes on the processor and for how long. Among preemptive priority policies, Longest Remaining Time First (LRTF) assigns the processor to the ready process with the largest residual execution time. Although LRTF is frequently mentioned in operating-systems curricula as the preemptive counterpart of Longest Job First (LJF), its mathematical structure, synchronization requirements, and real-time limitations are rarely examined together in a single coherent study. This paper provides a systematic treatment of LRTF covering formal definition, algorithmic behaviour, mathematical formulation, a fully worked numerical example, semaphore-based and monitor-based implementations, a hybrid semaphore–monitor architecture, and a comparative evaluation against Earliest Deadline First (EDF) and Rate Monotonic (RM). The paper distinguishes carefully between the scheduling policy and the synchronization mechanisms used to realize it. It shows that LRTF is not equivalent to LJF, Shortest Remaining Time First (SRTF), EDF, or RM, and that preemption alone does not confer deadline guarantees. A reproducible simulation design is presented, and all numerical results are labelled as illustrative unless explicitly stated otherwise. The paper concludes that LRTF is pedagogically valuable and can be implemented correctly with either semaphores or monitors, but that it is unsuitable as a hard real-time policy because it ignores deadlines and periods. EDF and RM remain the analytically grounded choices for periodic real-time workloads under their stated assumptions.

Keywords: LRTF; CPU scheduling; preemptive scheduling; semaphores; monitors; hybrid synchronization; EDF; rate monotonic; real-time systems; embedded systems.

[10/29/2026 1:51 AM] ..:

Table of Contents

1. Introduction
- **1.1** Background of CPU Scheduling
- **1.2** Importance of Process Scheduling
- **1.3** Scheduling Challenges in Embedded and Real-Time Systems
- **1.4** Research Problem and Motivation
- **1.5** Research Objectives and Questions
- **1.6** Scope and Limitations
- **1.7** Organization of the Paper

2. Theoretical Foundations
- **2.1** Processes, Threads, and CPU Bursts
- **2.2** Remaining Execution Time and Priorities
- **2.3** Preemptive and Non-Preemptive Scheduling
- **2.4** Context Switching and Scheduling Decisions
- **2.5** Performance Metrics
- **2.6** General-Purpose vs. Real-Time Scheduling

3. The LRTF Algorithm
- **3.1** Formal Definition
- **3.2** Core Operating Principle
- **3.3** Preemption and Tie-Breaking
- **3.4** Pseudocode and Complexity
- **3.5** Advantages, Disadvantages, and Fairness

4. Mathematical Formulation
- **4.1** Variables and Assumptions
- **4.2** Selection and Completion
- **4.3** Performance Metrics
- **4.4** Objective Function

5. LRTF Scheduling Example
- **5.1** Task Set and Assumptions
- **5.2** Step-by-Step Trace
- **5.3** Gantt Chart and Metric Calculations
- **5.4** Verification and Tie-Breaking Sensitivity

6. Implementation Using Semaphores
- **6.1** Semaphore Concepts
- **6.2** Shared Scheduling Data and Critical Sections
- **6.3** Architecture and Pseudocode

6.4 Testing Strategy and Limitations

7. Implementation Using Monitors

7.1 Monitor Concepts

7.2 Condition Variables and Ready-Queue Management

7.3 Architecture and Pseudocode

7.4 Correctness and Comparison with Semaphores

8. Hybrid Semaphore–Monitor Implementation

8.1 Motivation and Responsibility Assignment

8.2 Lock Ordering and Deadlock Prevention

8.3 Architecture and Pseudocode

8.4 Testing and Overhead

9. Comparative Analysis of Synchronization Approaches

10. Comparison with EDF and RM

10.1 Policy Definitions and Priority Assignment

10.2 Schedulability and Overhead

10.3 Application Scenarios

11. Experimental Design and Evaluation [10/29/2026 1:51 AM] ..:

11.1 Objectives and Environment

11.2 Task-Set Assumptions and Procedure

11.3 Illustrative Results and Discussion

11.4 Threats to Validity

12. Correctness, Limitations, and Practical Considerations

13. Final Discussion and Conclusion

14. References

List of Figures

Figure 1 CPU scheduling overview
Figure 2 Process-state diagram
Figure 3 LRTF scheduling architecture
Figure 4 LRTF decision-making flowchart
Figure 5 LRTF worked-example Gantt chart
Figure 6 Remaining execution time over time
Figure 7 Preemption illustration
Figure 8 Semaphore synchronization architecture
Figure 9 Semaphore-based flowchart
Figure 10 Monitor architecture
Figure 11 Monitor-based flowchart
Figure 12 Hybrid synchronization architecture
Figure 13 Hybrid implementation flowchart
Figure 14 LRTF–EDF–RM conceptual comparison
Figure 15 Example performance comparison chart
Figure 16 Experimental workflow
Figure 17 Correctness verification workflow
Figure 18 Overall project architecture and findings

List of Tables

Table 1 Research objectives and questions
Table 2 Scheduling terminology
Table 3 Symbols and mathematical notation
Table 4 Scheduling algorithm characteristics
Table 5 LRTF example task set
Table 6 Step-by-step remaining-time trace
Table 7 LRTF execution intervals
Table 8 Performance metric calculations
Table 9 Semaphore operations
Table 10 Monitor components
Table 11 Hybrid architecture responsibilities
Table 12 Synchronization-method comparison
Table 13 LRTF vs. EDF vs. RM
Table 14 Experimental parameters
Table 15 Illustrative simulation results
Table 16 Correctness and verification checklist
Table 17 Limitations and mitigation strategies
Table 18 Summary of findings and future work

List of Mathematical Symbols and Abbreviations

[10/29/2026 1:51 AM] ..:

Table 3: Symbols and mathematical notation

Symbol / Abbreviation	Meaning
P, P_i	Process (or task) and the i-th process
r_i	Arrival (release) time of P_i
e_i	Total required CPU execution time (burst length) of P_i
$R_i(t)$	Remaining execution time of P_i at time t
C_i	Completion time of P_i
T_i	Turnaround time of P_i
W_i	Waiting time of P_i
Rsp_i	Response time of P_i
U	CPU utilization
X	Throughput (completed processes per time unit)
D_i	Relative deadline of P_i
M	Number of deadline misses
ρ	Deadline-miss ratio
O_s	Scheduling overhead per decision
LRTF	Longest Remaining Time First
LJF	Longest Job First
SRTF	Shortest Remaining Time First
EDF	Earliest Deadline First
RM	Rate Monotonic
FCFS	First-Come, First-Served
RTOS	Real-Time Operating System

CHAPTER 1

Introduction

1.1 Background of CPU Scheduling

An operating system must decide, whenever the processor becomes available, which of the ready processes should execute next. This decision is made by the CPU scheduler, also called the short-term scheduler. The scheduler selects a process from the ready queue and dispatches it to the CPU. Scheduling is necessary because the number of runnable processes is usually larger than the number of processors, and because processes alternate between CPU bursts and I/O waits. A well-designed scheduling policy can improve responsiveness, throughput, fairness, and resource utilization.

The classical scheduling problem is defined over a set of processes, each characterized by an arrival time, an execution requirement, and possibly a deadline or priority. The scheduler produces a sequence of execution intervals, and the quality of that sequence is judged by metrics such as waiting time, turnaround time, response time, CPU utilization, and throughput. Different policies optimize different metrics, and no single policy is universally optimal. This trade-off is the central reason why operating systems expose multiple scheduling classes and why real-time systems use specialized schedulers.

[10/29/2026 1:51 AM] ..:

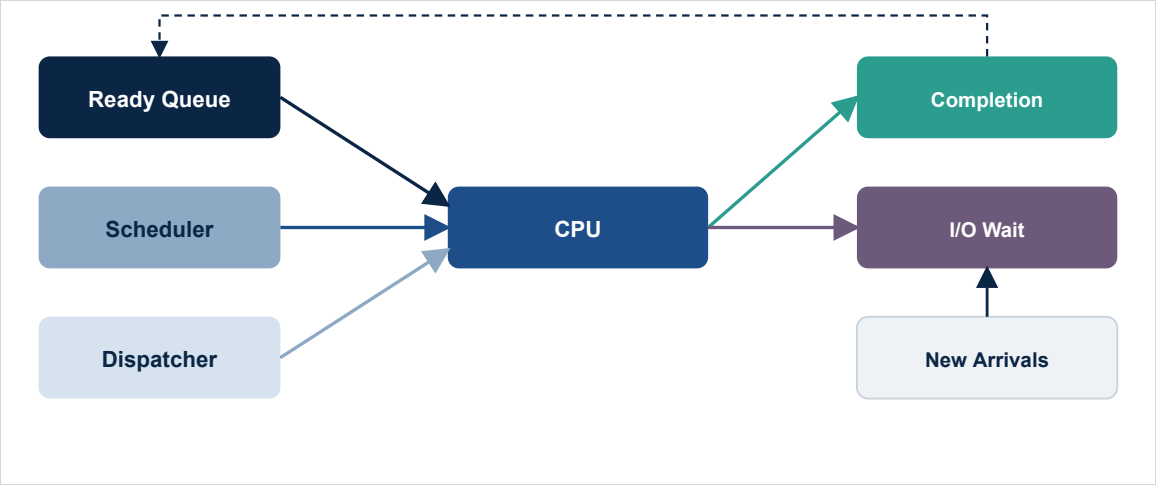


Figure 1: CPU scheduling overview. Ready processes are selected by the scheduler, dispatched to the CPU, and eventually either complete or enter an I/O wait, after which they return to the ready queue.

1.2 Importance of Process Scheduling in Operating Systems

Scheduling affects every user-visible aspect of system performance. In interactive systems, response time determines perceived responsiveness. In batch systems, throughput and turnaround time determine productivity. In real-time systems, missing a deadline can be catastrophic. Scheduling also affects fairness: a policy that favours long jobs may starve short jobs, while a policy that favours short jobs may delay long ones. The scheduler must therefore balance competing objectives under uncertain workloads and limited information about future arrivals.

[10/29/2026 1:51 AM] ..:
The CPU scheduler is also intimately connected to synchronization. Preemptive scheduling can interrupt a process in the middle of a critical section, so the kernel must protect shared scheduling data structures.

The choice of synchronization mechanism—semaphores, monitors, or a combination—affects correctness, readability, and overhead. This paper treats scheduling policy and synchronization mechanism as separate concerns, because conflating them is a common source of design errors.

1.3 Scheduling Challenges in Embedded and Real-Time Systems

Embedded systems often operate under severe resource constraints: limited memory, limited CPU power, and strict power budgets. Real-time embedded systems add timing constraints. A hard real-time task must complete before its deadline in every execution; a soft real-time task may occasionally miss a deadline with degraded quality. Scheduling policies for such systems must be predictable and analysable, not merely empirically fast.

Many embedded systems use fixed-priority preemptive scheduling, often Rate Monotonic (RM), because it is simple to implement and has a well-understood utilization bound. Dynamic-priority policies such as Earliest Deadline First (EDF) can achieve higher utilization but require more careful implementation. LRTF, by contrast, is not a deadline-based policy. It prioritizes the process with the largest remaining execution time, which can produce long waiting times for short tasks and is therefore generally unsuitable for hard real-time workloads. Nevertheless, LRTF is important pedagogically because it illustrates the difference between preemption and deadline-awareness, and because it provides a useful counterexample when discussing real-time schedulability.

1.4 Research Problem and Motivation

The research problem addressed in this paper is the absence of a unified treatment of LRTF that connects its formal definition, mathematical properties, synchronization requirements, and real-time limitations. Many textbook treatments mention LRTF only briefly, often as “the preemptive version of LJF,” without examining its behaviour under simultaneous arrivals, tie-breaking, starvation, or implementation overhead. Moreover, discussions of semaphores and monitors rarely use LRTF as a concrete case study, even though the scheduler's ready queue is a natural shared data structure that requires synchronization.

The motivation for this study is threefold. First, a rigorous mathematical formulation of LRTF clarifies its decision rule and performance metrics. Second, a side-by-side implementation using semaphores, monitors, and a hybrid design demonstrates the practical consequences of different synchronization choices. Third, a careful comparison with EDF and RM corrects the misconception that any preemptive policy is suitable for real-time systems.

1.5 Research Objectives and Questions

The objectives and questions of this study are summarized in Table 1.

[10/29/2026 1:51 AM] ..:

Table 1: Research objectives and questions

Objective	Research question
O1: Define LRTF formally and distinguish it from related policies.	Q1: How does LRTF differ from LJF, SRTF, EDF, and RM?
O2: Formulate LRTF mathematically and define its performance metrics.	Q2: Which equations govern process selection and metric computation?
O3: Demonstrate LRTF with a fully worked example.	Q3: What schedule does LRTF produce for a given task set, and how sensitive is it to tie-breaking?
O4: Implement LRTF using semaphores and monitors.	Q4: Which shared resources require synchronization, and how do semaphores and monitors differ in practice?
O5: Design a hybrid semaphore–monitor architecture.	Q5: Can both mechanisms be combined meaningfully without redundant locking or deadlock?
O6: Compare LRTF with EDF and RM.	Q6: Under what conditions, if any, is LRTF appropriate for real-time or embedded systems?
O7: Design a reproducible evaluation methodology.	Q7: How can LRTF behaviour be evaluated without fabricating empirical claims?

1.6 Scope and Limitations

This paper focuses on single-processor, preemptive scheduling of independent processes with known execution times. It assumes that the scheduler has complete knowledge of remaining execution times, which is an idealization: in real systems, remaining times must be estimated. The paper does not address multiprocessor scheduling, energy-aware scheduling, or scheduling under uncertainty. The implementations are presented as pseudocode and reference simulations; they have not been benchmarked on a specific hardware platform. All numerical examples are illustrative and are clearly labelled as such.

1.7 Organization of the Paper

Chapter 2 establishes theoretical foundations. Chapter 3 defines LRTF and analyses its properties. Chapter 4 presents the mathematical formulation. Chapter 5 provides a worked example. Chapters 6–8 present semaphore-based, monitor-based, and hybrid implementations. Chapter 9 compares the synchronization approaches. Chapter 10 compares LRTF with EDF and RM. Chapter 11 describes the experimental design. Chapter 12 discusses correctness and limitations. Chapter 13 concludes the paper.

CHAPTER 2

Theoretical Foundations

2.1 Processes, Threads, and CPU Bursts

A process is an executing program with its own address space, resources, and execution state. A thread is a lighter-weight execution unit within a process, sharing the process's address space with other threads. From the scheduler's perspective, the unit of scheduling is usually a thread or a kernel-level schedulable entity, although the terms “process” and “task” are often used interchangeably in scheduling theory.

Processes alternate between CPU bursts and I/O bursts. A CPU burst is a period of uninterrupted computation; an I/O burst is a period of waiting for external data. The CPU-burst duration distribution is typically skewed: many short bursts and a few long bursts. Scheduling policies that favour short bursts, such as Shortest Job First (SJF) and Shortest Remaining Time First (SRTF), reduce average waiting time but may starve long bursts. LRTF does the opposite: it favours long remaining bursts, which tends to delay short tasks and can produce high average waiting times.

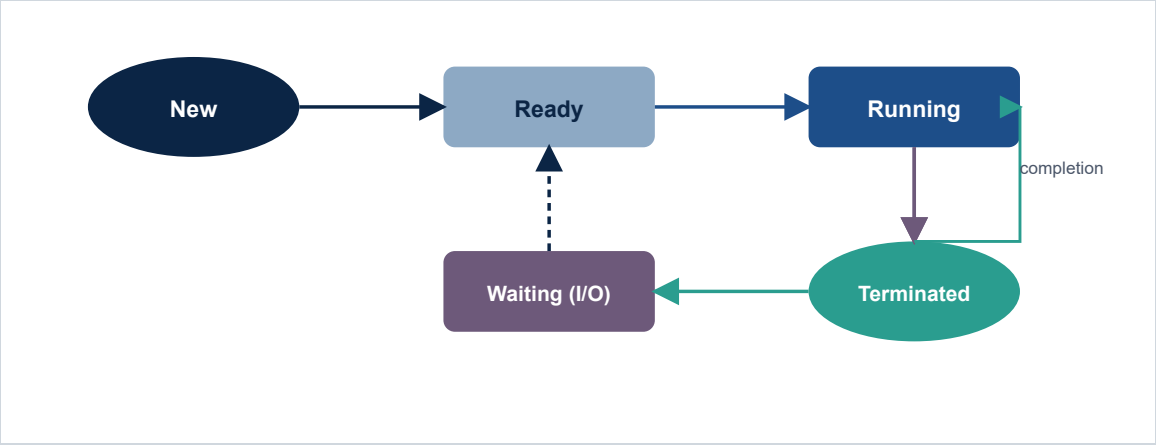


Figure 2: Process-state diagram. A process moves from New to Ready, then to Running when dispatched. It may block into Waiting and later return to Ready, or terminate after completion.

2.2 Remaining Execution Time and Priorities

The remaining execution time of a process P_i at time t, denoted $R_i(t)$, is the amount of CPU time still required for the process to complete. Immediately after arrival, $R_i(r_i) = e_i$. While the process executes, its remaining time decreases at unit rate; while it is preempted or waiting, its remaining time stays constant. When $R_i(t) = 0$, the process has completed.

A priority is a value that determines the order in which ready processes are considered. Priorities may be static (assigned once at creation) or dynamic (recomputed at each scheduling decision). LRTF uses a dynamic priority equal to the remaining execution time: higher remaining time means higher priority. This is the opposite of SRTF, which gives higher priority to smaller remaining times. It is also different from EDF, where priority is the absolute deadline, and from RM, where priority is a fixed function of the period.

2.3 Preemptive and Non-Preemptive Scheduling

In non-preemptive scheduling, a process, once dispatched, runs until it completes or voluntarily blocks. In

preemptive scheduling, the scheduler may interrupt a running process when a higher-priority process becomes ready. Preemption is essential for responsiveness in interactive and real-time systems, but it introduces context-switch overhead and complicates synchronization.

[10/29/2026 1:51 AM] ..:
LRTF is preemptive: if a newly arrived process has a larger remaining execution time than the currently running process, the scheduler preempts the running process and dispatches the newcomer. Preemption can also occur at other scheduling points, such as when a process completes or when a blocked process becomes ready. The exact set of scheduling points depends on the kernel implementation.

2.4 Context Switching and Scheduling Decisions

A context switch is the act of saving the state of the current process and restoring the state of the next process. It consumes CPU time and may invalidate caches, so frequent preemption can reduce effective throughput. Scheduling decisions occur when a process terminates, when a process blocks, when a process becomes ready after I/O, or when a new process arrives. In preemptive systems, timer interrupts also create scheduling points.

2.5 Performance Metrics

Scheduling performance is evaluated using several metrics. Turnaround time is the elapsed time from arrival to completion. Waiting time is the total time spent in the ready queue. Response time is the time from arrival to the first dispatch. CPU utilization is the fraction of time the CPU is busy. Throughput is the number of completed processes per unit time. These metrics are defined formally in Chapter 4. Table 2 summarizes the terminology.

Table 2: Scheduling terminology

Term	Definition
Arrival time	Time at which a process becomes ready to execute.
Execution time	Total CPU time required by a process to complete.
Remaining time	CPU time still required at a given instant.
Preemption	Interruption of a running process so that another can execute.
Context switch	Saving and restoring processor state between processes.
Dispatch	Assignment of the CPU to the selected process.
Starvation	Indefinite postponement of a process because higher-priority processes keep arriving.
Turnaround time	Completion time minus arrival time.
Waiting time	Turnaround time minus execution time.
Response time	Time from arrival to first dispatch.

2.6 General-Purpose vs. Real-Time Scheduling

General-purpose scheduling aims to provide good average performance and fairness across diverse workloads. Real-time scheduling aims to guarantee that timing constraints are met. Real-time policies are typically analysed under a periodic task model, where each task has a period, a worst-case execution time, and a relative deadline. EDF and RM are the canonical real-time policies. LRTF is not defined in terms of

deadlines or periods, so it does not inherit the schedulability guarantees of EDF or RM. Table 4 contrasts the characteristics of several policies.

[10/29/2026 1:51 AM] ..:

Table 4: Scheduling algorithm characteristics

Algorithm	Priority type	Preemptive	Key parameter	Typical use
FCFS	Arrival order	No	Arrival time	Simple batch systems
SJF	Static (burst)	No	Execution time	Batch, known workloads
SRTF	Dynamic (residual)	Yes	Remaining time	General-purpose, interactive
LJF	Static (burst)	No	Execution time	Pedagogical, rare in practice
LRTF	Dynamic (residual)	Yes	Remaining time	Pedagogical, counterexample
RM	Static (period)	Yes	Period	Periodic hard real-time
EDF	Dynamic (deadline)	Yes	Absolute deadline	Periodic real-time, higher utilization

CHAPTER 3

The LRTF Algorithm

3.1 Formal Definition

Let $\mathcal{P} = \{P_1, \ldots, P_n\}$ be a set of processes. At any scheduling instant t, let $\mathcal{R}(t)$ be the set of ready processes. LRTF selects the process

$$P_{\text{next}} = \underset{P_i \in \mathcal{R}(t)}{\arg\max}\, R_i(t) \tag{1}$$

with ties broken by a stated rule, typically First-In, First-Out (FIFO) among processes with equal remaining time. Equation (1) defines the policy: the ready process with the largest residual execution time receives the processor. This is the preemptive form of Longest Job First (LJF), but it is not identical to LJF because the priority is recomputed dynamically rather than fixed at arrival.

3.2 Core Operating Principle

At each scheduling point, the scheduler examines the remaining execution time of every ready process. It selects the process with the maximum remaining time and dispatches it. If a new process arrives with a remaining time larger than that of the running process, the running process is preempted. If the running process completes or blocks, the scheduler again selects the ready process with the largest remaining time. Over time, LRTF tends to equalize completion times: the longest remaining process is repeatedly favoured, so shorter processes finish earlier, while long processes finish near the end. This behaviour is sometimes described as “anti-SRTF.”

3.3 Preemption and Tie-Breaking

Preemption in LRTF occurs when a newly ready process has a strictly larger remaining time than the currently running process. If the new process has an equal remaining time, the tie-breaking rule determines whether preemption occurs. FIFO is a common choice: the process that arrived earlier has priority. Other rules, such as priority by process identifier, are possible but must be stated explicitly because they affect the resulting schedule.

Simultaneous arrivals are handled by inserting all newly arrived processes into the ready queue before making the scheduling decision. If two processes arrive at the same instant with the same remaining time, the tie-breaking rule is applied. The choice of tie-breaking rule can change completion times and metric values, so it must be documented in any worked example or implementation.

3.4 Pseudocode and Complexity

The core LRTF decision procedure is shown in pseudocode below. It assumes a ready queue that supports extracting the maximum remaining time.

[10/29/2026 1:51 AM] ..:

Algorithm 1: LRTF scheduling decision

```
function LRTF_Select(ReadySet):
  if ReadySet is empty: return Idle
  best ← first element of ReadySet
  for each P in ReadySet:
    if R(P) > R(best): best ← P
    else if R(P) == R(best) and tie_break(P, best): best ← P
  return best
```

If the ready set is stored as an unsorted list, each decision costs $O(n)$. If a max-heap keyed by remaining time is used, selection costs $O(1)$ and updates after preemption or completion cost $O(\log n)$. LRTF's computational complexity is therefore comparable to other priority-based schedulers; the policy itself is not computationally expensive. The practical overhead comes from frequent preemption and context switching, not from the selection step.

3.5 Advantages, Disadvantages, and Fairness

LRTF is simple to define and easy to implement in a simulator. It guarantees that all processes eventually complete if the CPU is not overloaded, and it tends to make long jobs finish at approximately the same time. However, it has significant disadvantages. Short processes can be repeatedly preempted by newly arriving long processes, leading to high waiting times and poor responsiveness. Average waiting time is typically much worse than under SRTF. LRTF also ignores deadlines entirely, so it provides no real-time guarantees. Finally, its fairness properties are debatable: although no process is permanently starved under finite arrivals, short processes can suffer excessive delay.

A common misconception is that LRTF is starvation-free in all cases. Under a continuous stream of long arrivals, short processes can be postponed indefinitely. Therefore, claims of universal fairness should be treated with caution and qualified by the arrival model.

CHAPTER 4

Mathematical Formulation

4.1 Variables and Assumptions

We assume a single processor and a set of n independent processes. Each process P_i has an arrival time r_i, an execution requirement $e_i > 0$, and optionally a relative deadline D_i. The scheduler is preemptive and makes decisions at arrival, completion, and blocking instants. Context-switch overhead is initially ignored and later modelled separately.

4.2 Selection and Completion

The remaining execution time evolves as

$$\frac{dR_i(t)}{dt} = \begin{cases} -1 & \text{if } P_i \text{ is running at } t, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

In discrete-event simulation, this differential equation is replaced by unit-step updates. The completion time C_i is the first time at which $R_i(t) = 0$. The selection rule is given by (1).

[10/29/2026 1:51 AM] ..:

4.3 Performance Metrics

The turnaround time of P_i is

$$T_i = C_i - r_i \tag{3}$$

The waiting time is the total time spent in the ready queue:

$$W_i = T_i - e_i \tag{4}$$

This definition assumes no time spent in I/O or other non-ready states. If a process blocks, the waiting time must exclude blocked intervals. The response time is the time from arrival to first dispatch:

$$\text{Rsp}_i = \text{first dispatch time of } P_i - r_i \tag{5}$$

CPU utilization over an interval $[0, T]$ is

$$U = \frac{\sum_i e_i}{T} \tag{6}$$

and throughput is

$$X = \frac{n}{T} \tag{7}$$

Average waiting time and average turnaround time are

$$\overline{W} = \frac{1}{n} \sum_{i=1}^{n} W_i \tag{8}$$

and

$$\overline{T} = \frac{1}{n} \sum_{i=1}^{n} T_i \tag{9}$$

respectively. For real-time evaluation, the deadline-miss count is

$$M = \sum_{i} \mathbb{I}\left(C_i > r_i + D_i\right) \tag{10}$$

where $\mathbb{I}(\cdot)$ is the indicator function. The deadline-miss ratio is $\rho = M / n$. Scheduling overhead is modelled as

$$O_{\text{total}} = N_{\text{dec}} \times O_s + N_{\text{cs}} \times O_c \tag{11}$$

[10/29/2026 1:51 AM] ..:
where N_{dec} is the number of scheduling decisions, O_s is the per-decision cost, N_{cs} is the number of context switches, and O_c is the per-switch cost.

4.4 Objective Function

A general evaluation function can combine normalized metrics:

$$J = w_1 \frac{\overline{W}}{W_{\text{ref}}} + w_2 \frac{\rho}{\rho_{\text{ref}}} + w_3 \frac{O_{\text{total}}}{O_{\text{ref}}} \tag{12}$$

where w_1, w_2, w_3 are application-specific weights. This function is not a universal measure of “goodness”; it must be instantiated for a particular system. No single metric is universally applicable, because the importance of waiting time versus deadline misses versus overhead depends on the workload and the system's requirements.

CHAPTER 5

LRTF Scheduling Example

5.1 Task Set and Assumptions

Consider the four-process task set in Table 5. All times are in abstract time units. The tie-breaking rule is FIFO: among processes with equal remaining time, the one that arrived earlier is selected. If arrival times are equal, the process with the smaller identifier is selected.

Table 5: LRTF example task set

Process	Arrival time r_i	Execution time e_i
P1	0	10
P2	1	4
P3	2	6
P4	3	2

5.2 Step-by-Step Trace

At $t = 0$, only P1 is ready, so P1 runs. At $t = 1$, P2 arrives with remaining time 4. P1 has remaining time 9, so P1 continues. At $t = 2$, P3 arrives with remaining time 6. P1 has remaining time 8, so P1 continues. At $t = 3$, P4 arrives with remaining time 2. P1 still has the largest remaining time (7), so P1 continues. P1 runs until it completes at $t = 10$, because no arriving process has a remaining time larger than P1's remaining time. After P1 completes, the ready set contains P2 (remaining 4), P3 (remaining 6), and P4 (remaining 2). P3 is selected. P3 runs from $t = 10$ to $t = 16$. Then P2 runs from 16 to 20, and P4 runs from 20 to 22. The complete trace is shown in Table 6 and the execution intervals in Table 7.

[10/29/2026 1:51 AM] ..:

Table 6: Step-by-step remaining-time trace

Time	Event	Ready set (remaining times)	Selected
0	P1 arrives	P1(10)	P1
1	P2 arrives	P1(9), P2(4)	P1
2	P3 arrives	P1(8), P2(4), P3(6)	P1
3	P4 arrives	P1(7), P2(4), P3(6), P4(2)	P1
10	P1 completes	P2(4), P3(6), P4(2)	P3
16	P3 completes	P2(4), P4(2)	P2
20	P2 completes	P4(2)	P4
22	P4 completes	—	Idle

Table 7: LRTF execution intervals

Interval	Running process	Duration
[0, 10)	P1	10
[10, 16)	P3	6
[16, 20)	P2	4
[20, 22)	P4	2

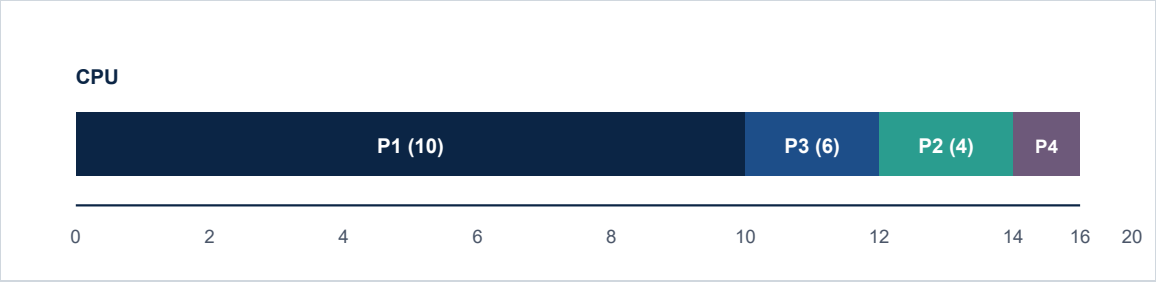


Figure 5: LRTF worked-example Gantt chart for the task set in Table 5. P1 runs uninterrupted because no arriving process has a larger remaining time. P3, P2, and P4 execute in decreasing order of remaining time after P1 completes.



Figure 6: Remaining execution time over time for the worked example. P1's remaining time decreases linearly while it runs; the other processes' remaining times stay constant until they are dispatched.

5.3 Gantt Chart and Metric Calculations

The completion times are $C_1 = 10$, $C_2 = 20$, $C_3 = 16$, and $C_4 = 22$. The turnaround times are 10, 19, 14, and 19. The waiting times are 0, 15, 8, and 17. The average waiting time is $(0 + 15 + 8 + 17) / 4 = 10$, and the average turnaround time is $(10 + 19 + 14 + 19) / 4 = 15.5$. These values are summarized in Table 8.

Table 8: Performance metric calculations

Process	Arrival	Execution	Completion	Turnaround	Waiting	Response
P1	0	10	10	10	0	0
P2	1	4	20	19	15	15
P3	2	6	16	14	8	8
P4	3	2	22	19	17	17
Average	—	—	—	**15.5**	**10.0**	**10.0**

5.4 Verification and Tie-Breaking Sensitivity

Verification: the sum of execution times is $10 + 4 + 6 + 2 = 22$, which equals the final completion time. The CPU is busy for the entire interval $[0, 22]$, so utilization is 100%. Each process completes exactly once, and no process completes before its arrival. [10/29/2026 1:51 AM] ..: The waiting times satisfy $W_i = C_i - r_i - e_i$, confirming internal consistency.

Tie-breaking sensitivity: if the tie-breaking rule were “smallest identifier first” instead of FIFO, the schedule in this example would not change, because no ties occur at scheduling points. To demonstrate sensitivity, consider a modified task set in which P2 arrives at $t = 10$ with remaining time 6, exactly equal to P3's remaining time. Under FIFO, P3 would be selected; under smallest-identifier-first, P2 would be selected. This would change completion times and metrics. The example therefore shows that tie-breaking must be stated explicitly and applied consistently.

CHAPTER 6

Implementation Using Semaphores

6.1 Semaphore Concepts

A semaphore is a synchronization primitive introduced by Dijkstra. A counting semaphore has an integer value and supports two atomic operations: `wait(S)` (also called `P` or `down`) and `signal(S)` (also called `V` or `up`). `wait(S)` decrements the value if it is positive; otherwise, the calling thread blocks. `signal(S)` increments the value and wakes a blocked thread if one exists. A binary semaphore, or mutex, has values 0 and 1 and is used for mutual exclusion.

Table 9: Semaphore operations

Operation	Semantics	Typical use
`wait(S)`	Atomically decrement if $S > 0$; otherwise block.	Acquire a resource or enter a critical section.
`signal(S)`	Atomically increment and wake a blocked waiter if any.	Release a resource or leave a critical section.
Binary semaphore	Value 0 or 1.	Mutual exclusion.
Counting semaphore	Non-negative integer.	Managing multiple identical resources.

6.2 Shared Scheduling Data and Critical Sections

In a semaphore-based LRTF implementation, the shared resources are the ready queue, the process table, and the remaining-time fields. The ready queue must be updated atomically when a process arrives, completes, or is preempted. The process table must be updated when a process's remaining time changes or when its state changes. A binary semaphore `mutex` protects these structures. A second semaphore `work_available` signals that the ready queue is non-empty and that a worker thread may dispatch a process.

It is crucial to understand that the semaphore does not implement LRTF. The semaphore provides mutual exclusion and signalling; the LRTF policy is implemented by the selection function that scans the ready queue and chooses the process with the largest remaining time. Conflating the policy with the synchronization mechanism is a conceptual error.

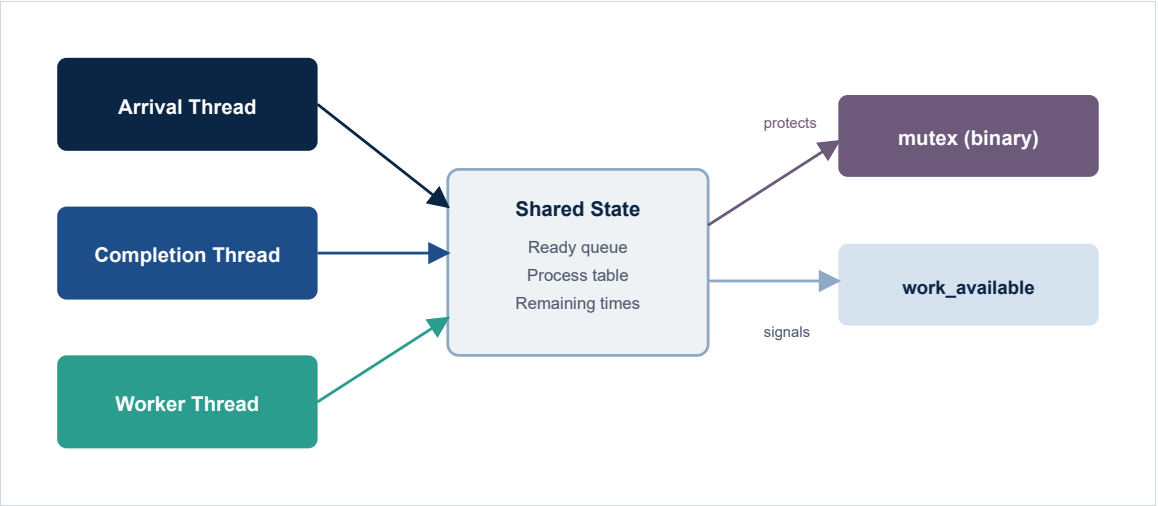


Figure 8: Semaphore synchronization architecture. A binary mutex protects the shared scheduling data, while a signalling semaphore coordinates worker dispatch.

6.3 Architecture and Pseudocode

The architecture has three logical threads: an arrival thread, a completion thread, and a worker thread. The arrival thread inserts new processes into the ready queue under `mutex` and signals `work_available`. The completion thread removes the completed process, updates the process table, and signals `work_available` if the ready queue is non-empty. The worker thread waits on `work_available`, acquires `mutex`, selects the process with the largest remaining time, releases `mutex`, and executes the process for a quantum or until preemption.

Pseudocode: semaphore-based LRTF worker

[10/29/2026 1:59 AM] ..: `loop:`

```
  wait(work_available)
  wait(mutex)
  if ready_queue empty:
    signal(mutex); continue
  P ← argmax_{Q ∈ ready_queue} R(Q)
  remove P from ready_queue
  signal(mutex)
  run P for one time unit
  wait(mutex)
  R(P) ← R(P) − 1
  if R(P) == 0: mark P completed
  else: insert P into ready_queue
  signal(mutex)
  signal(work_available)
```

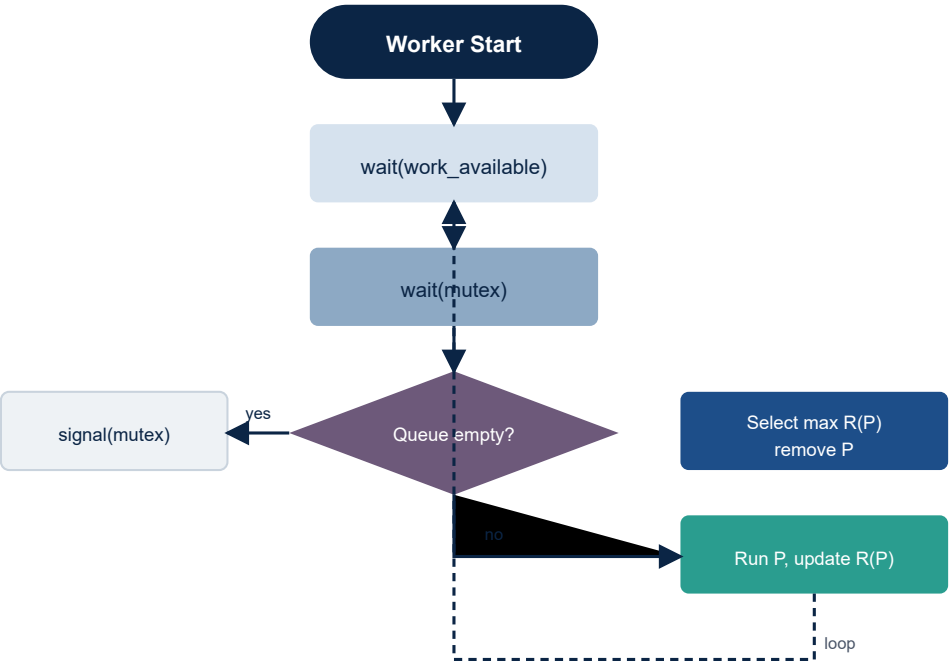


Figure 9: Semaphore-based LRTF worker flowchart. The worker waits for work, acquires the mutex, selects the process with maximum remaining time, releases the mutex, and updates the shared state after execution.

6.4 Testing Strategy and Limitations

Testing a semaphore-based LRTF implementation should include unit tests for the selection function, stress tests with concurrent arrivals and completions, and race-detection tests under randomized schedules. [10/29/2026 1:59 AM] ..: Common bugs include forgetting to signal `work_available` after inserting a process, updating remaining time without holding `mutex`, and deadlocking by acquiring locks in inconsistent order. The main limitation of semaphores is that they are low-level: the programmer must ensure that every path through the code releases the mutex, and condition waiting must be implemented manually. This increases the risk of errors compared with monitors.

CHAPTER 7

Implementation Using Monitors

7.1 Monitor Concepts

A monitor is a high-level synchronization construct that encapsulates shared data and the procedures that operate on that data. Only one thread can execute inside a monitor at a time, so mutual exclusion is provided implicitly. Monitors also provide condition variables, which allow threads to wait until a condition becomes true. The operations on a condition variable are `wait` and `signal` (or `notify`). `wait` releases the monitor lock and blocks the calling thread; `signal` wakes one waiting thread, which must reacquire the monitor lock before returning.

Table 10: Monitor components

Component	Purpose
Monitor lock	Ensures mutual exclusion among monitor procedures.
Shared data	Ready queue, process table, remaining times.
Condition variable `not_empty`	Signals that the ready queue is non-empty.
Condition variable `not_full`	Signals that the ready queue has space (if bounded).
Procedure `insert(P)`	Adds a process to the ready queue.
Procedure `select()`	Removes and returns the process with maximum remaining time.
Procedure `update(P, r)`	Updates the remaining time of a process.

7.2 Condition Variables and Ready-Queue Management

In a monitor-based LRTF implementation, the ready queue and process table are encapsulated inside a `SchedulerMonitor`. The `insert` procedure adds a process and signals `not_empty`. The `select` procedure waits on `not_empty` if the queue is empty, then removes the process with the largest remaining time. The `update` procedure modifies remaining time and either reinserts the process or marks it completed. Because the monitor lock is held during these procedures, the shared state remains consistent.

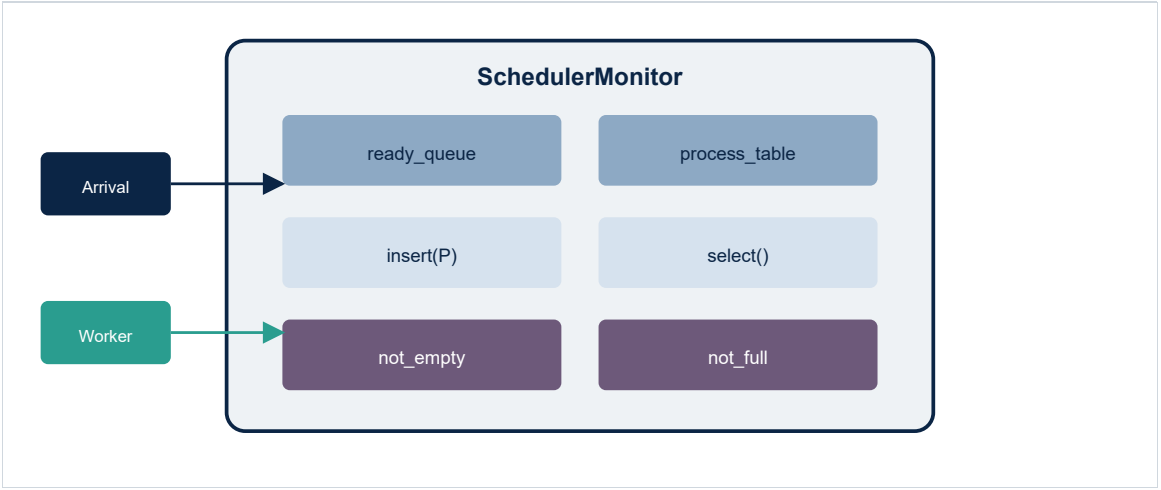


Figure 10: Monitor architecture. Shared scheduling data and procedures are encapsulated inside the monitor; condition variables provide waiting and signalling.

7.3 Architecture and Pseudocode

Pseudocode: monitor-based LRTF

```
monitor SchedulerMonitor:
  ready_queue: priority queue keyed by remaining time
  process_table: map from PID to process record
  condition not_empty

  procedure insert(P):
    add P to ready_queue
    signal(not_empty)

  procedure select() returns P:
    while ready_queue is empty:
      wait(not_empty)
    P ← remove process with maximum R(P)
    return P

  procedure update(P, r):
    process_table[P].remaining ← r
    if r == 0: process_table[P].state ← completed
    else: insert(P)
```

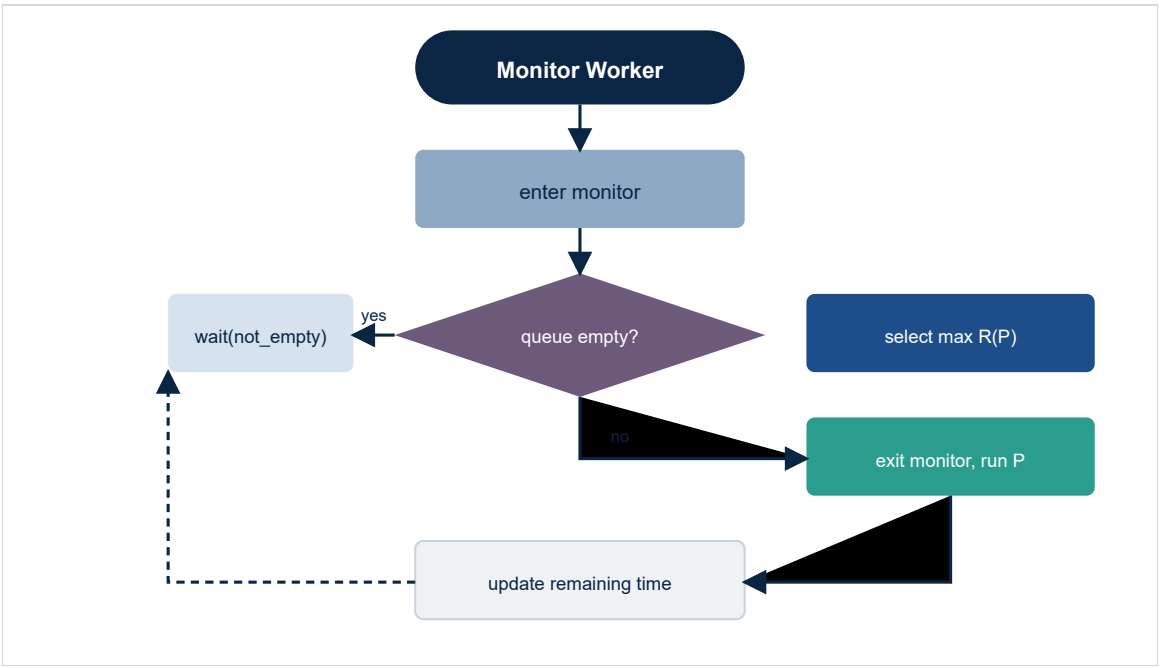


Figure 11: Monitor-based LRTF flowchart. The monitor provides implicit mutual exclusion; the worker waits on `not_empty` when the queue is empty and selects the maximum-remaining-time process when it is not.

7.4 Correctness and Comparison with Semaphores

Monitors reduce the risk of synchronization errors because mutual exclusion is enforced by the monitor

rather than by explicit lock/unlock calls. Condition variables are associated with the monitor, so waiting and signalling are type-checked and scoped. However, monitors are not a panacea: the programmer must still use `while` loops around condition waits to handle spurious wakeups, and must avoid signalling outside the monitor. In languages that do not natively support monitors, a monitor-like construct can be built from a mutex and condition variables, but it should not be described as a native monitor.

CHAPTER 8

Hybrid Semaphore–Monitor Implementation

8.1 Motivation and Responsibility Assignment

A hybrid design uses both semaphores and monitors, but assigns them distinct responsibilities. The monitor encapsulates the ready queue and the LRTF selection logic, because these are policy-related shared data structures that benefit from high-level mutual exclusion. Semaphores are used for event signalling between the scheduler and worker threads, because they provide efficient wake-up semantics without holding the monitor lock during execution. This division avoids redundant locking: the monitor lock protects shared state, and semaphores signal events.

[10/29/2026 1:59 AM] ..:

Table 11: Hybrid architecture responsibilities

Mechanism	Responsibility	Rationale
Monitor `SchedulerMonitor`	Protect ready queue and process table; implement `insert`, `select`, `update`.	High-level mutual exclusion; policy logic is localized.
Semaphore `work_available`	Signal that a process is ready for execution.	Efficient event notification; avoids busy waiting.
Semaphore `completion`	Signal that a process has completed.	Coordinates completion handling with the monitor.

8.2 Lock Ordering and Deadlock Prevention

Deadlock can arise if threads acquire the monitor lock and semaphore waits in inconsistent orders. The hybrid design enforces a strict lock ordering: a thread must never call `wait` on a semaphore while holding the monitor lock. Instead, the monitor procedure returns the selected process and releases the lock, after which the worker waits on `work_available`. This ordering prevents circular waits. The completion path similarly updates the monitor and then signals `completion` outside the monitor.

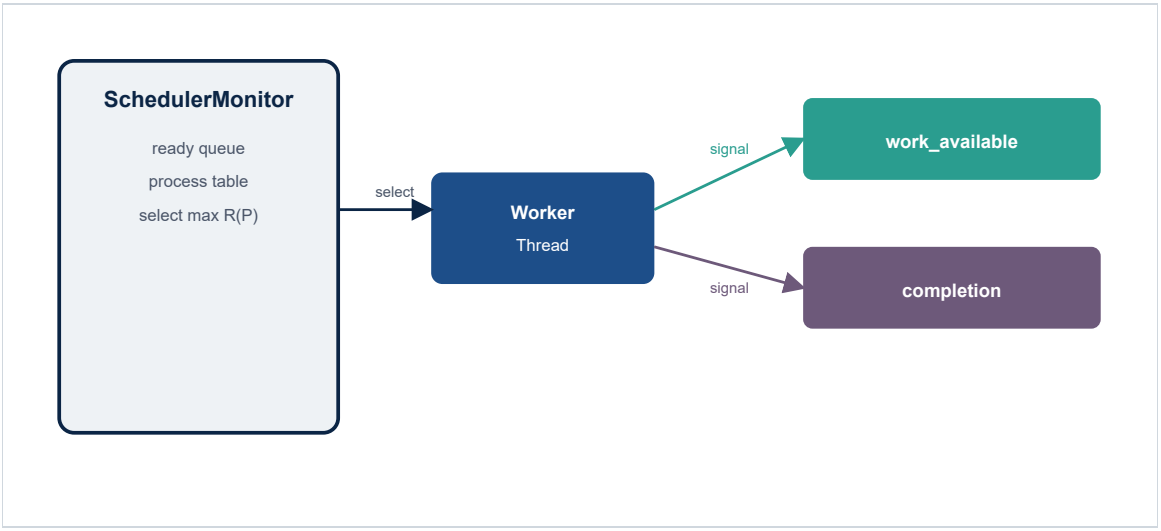


Figure 12: Hybrid synchronization architecture. The monitor owns the policy state, while semaphores signal work availability and completion events.

8.3 Architecture and Pseudocode

[10/29/2026 1:59 AM] ..:

Pseudocode: hybrid LRTF worker

```
loop:
  wait(work_available)
  P ← SchedulerMonitor.select()
  run P for one time unit
  SchedulerMonitor.update(P, R(P) - 1)
  if P completed: signal(completion)
  else: signal(work_available)
```

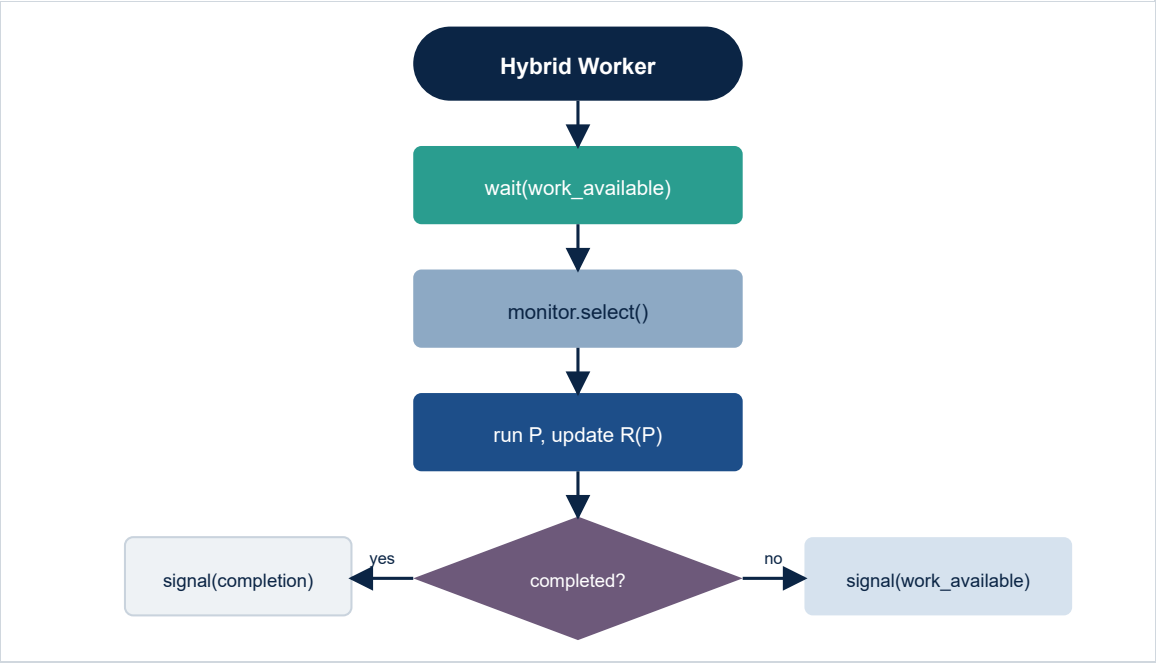


Figure 13: Hybrid implementation flowchart. The worker waits on a semaphore, uses the monitor to select and update state, and signals completion or further work.

8.4 Testing and Overhead

Testing should verify that the monitor lock is never held while waiting on a semaphore, that every completed process signals `completion` exactly once, and that `work_available` is signalled whenever the ready queue transitions from empty to non-empty. The hybrid design has slightly higher overhead than a pure monitor design because it uses both a monitor lock and semaphore operations, but the overhead is bounded and predictable. The main complexity is the need to reason about two synchronization mechanisms simultaneously.

[10/29/2026 1:59 AM] ..:

CHAPTER 9

Comparative Analysis of Synchronization Approaches

Table 12 compares the three synchronization designs along dimensions relevant to LRTF implementation. The assessments are qualitative and based on the design analysis in Chapters 6–8; they are not measured benchmark results.

Table 12: Synchronization-method comparison

Criterion	Semaphores	Monitors	Hybrid
Implementation complexity	Moderate; manual lock/unlock on every path.	Low; implicit mutual exclusion.	High; two mechanisms must be coordinated.
Readability	Low to moderate.	High; policy logic is encapsulated.	Moderate; responsibilities are separated.
Mutual exclusion	Correct if every critical section is protected.	Correct by construction.	Correct if lock ordering is enforced.
Race-condition risk	Higher; easy to forget a lock.	Lower; monitor lock is automatic.	Low if semaphores are used only for signalling.
Deadlock risk	Moderate; depends on lock ordering.	Low; single monitor lock.	Moderate; requires strict ordering.
Ease of testing	Requires stress tests and race detectors.	Easier; state is localized.	Requires both race and deadlock tests.
Performance overhead	Low.	Low to moderate.	Moderate.
Scalability	Good for simple designs.	Limited by single monitor lock.	Moderate; can be partitioned.
Educational suitability	Good for illustrating primitives.	Good for illustrating structured synchronization.	Good for illustrating separation of concerns.
Embedded suitability	Good when a small RTOS provides semaphores.	Depends on language/runtime support.	Feasible but adds code size.

The semaphore design is closest to the hardware and gives the programmer explicit control, but it demands discipline. The monitor design is easier to reason about because the shared state and the operations on it are encapsulated. The hybrid design is useful when event signalling and policy protection have different performance or structural requirements, but it is the most complex of the three. None of these mechanisms implements LRTF by itself; they only make the implementation correct under concurrency.

CHAPTER 10

Comparison with EDF and RM

10.1 Policy Definitions and Priority Assignment

EDF assigns priority according to absolute deadline: the ready task with the earliest deadline runs first. RM assigns fixed priorities according to period: shorter periods receive higher priorities. LRTF assigns priority according to remaining execution time: the task with the largest remaining time runs first. These policies use fundamentally different priority signals. LRTF is not a deadline-based policy and does not use period information.

Under the periodic task model, EDF is optimal for preemptive uniprocessor scheduling when deadlines equal periods: if any policy can meet all deadlines, EDF can. RM is optimal among fixed-priority policies, but its worst-case utilization bound is $n\left(2^{1/n}-1\right)$, which approaches $\ln 2 \approx 0.693$ as [10/29/2026 1:59 AM] ..: n grows. EDF can utilize up to 100% of the processor in the same model. LRTF has no such schedulability bound because it does not consider deadlines or periods.

10.2 Schedulability and Overhead

EDF and RM have well-defined schedulability tests under the Liu–Layland assumptions: periodic tasks, independent tasks, fixed execution times, deadlines equal to periods, and negligible context-switch overhead. LRTF has no equivalent test. Its behaviour depends on the arrival pattern and execution-time distribution, and it can delay short tasks arbitrarily under adversarial arrivals. In terms of runtime overhead, EDF requires maintaining deadlines and selecting the minimum deadline; RM requires a fixed-priority queue; LRTF requires maintaining remaining times and selecting the maximum. All three can be implemented in $O(\log n)$ per decision with appropriate data structures.

10.3 Application Scenarios

EDF is appropriate for periodic real-time systems where deadlines are known and the task set is analysable. RM is appropriate for fixed-priority systems with short periods and a conservative utilization bound. LRTF is appropriate only as a pedagogical example, a workload generator for stress testing, or a scheduling policy for non-real-time systems where long jobs should be completed together. It should not be used for hard real-time control loops, where deadline misses can be catastrophic. Table 13 summarizes the comparison.

Table 13: LRTF vs. EDF vs. RM

Property	LRTF	EDF	RM
Priority signal	Remaining execution time (max)	Absolute deadline (min)	Period (min)
Priority type	Dynamic	Dynamic	Static
Preemptive	Yes	Yes	Yes
Uses deadlines	No	Yes	Yes (indirectly via periods)
Schedulability test	None	Utilization ≤ 1 (periodic, implicit deadlines)	Utilization bound (Liu–Layland)
Real-time guarantees	None	Optimal under stated assumptions	Guaranteed up to the bound
Typical use	Pedagogical, counterexample	Periodic real-time systems	Fixed-priority RTOS

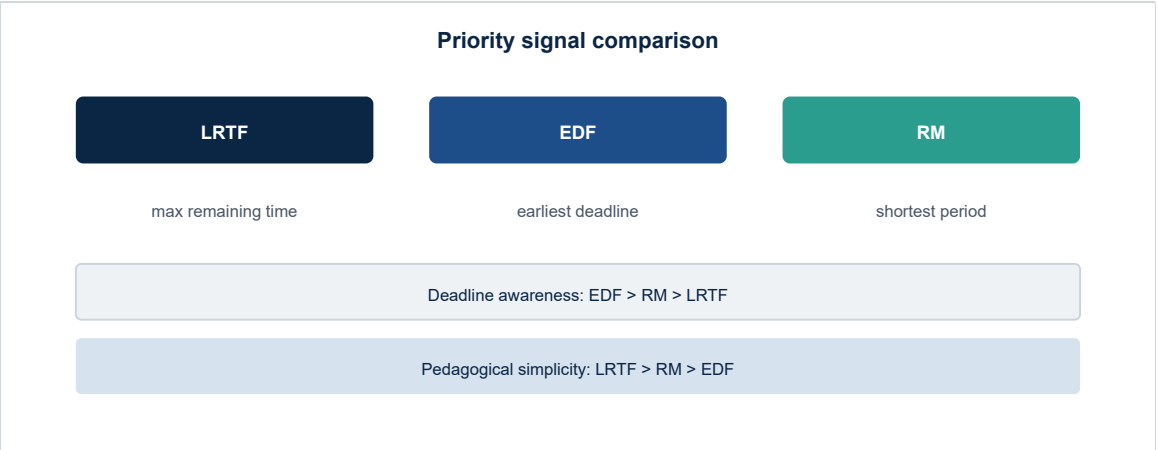


Figure 14: LRTF–EDF–RM conceptual comparison. The policies differ in the signal used to assign priority and in their degree of deadline awareness.

CHAPTER 11

Experimental Design and Evaluation

11.1 Objectives and Environment

The objective of the experimental design is to evaluate LRTF behaviour under controlled, reproducible conditions and to compare it with EDF and RM on a common task set. The design is intended for a discrete-event simulator implemented in Python 3.x on a standard desktop computer. No hardware performance measurements are claimed; all results are generated by the simulator and are labelled as illustrative.

11.2 Task-Set Assumptions and Procedure

We assume a periodic task set with $n = 4$ tasks. Each task τ_i has a period T_i, a worst-case execution time E_i, and an implicit deadline $D_i = T_i$. The task parameters are chosen so that the total utilization is below 1. The simulator schedules jobs for a hyperperiod and records completion times, deadline misses, and context switches. For LRTF, remaining times are known to the scheduler; for EDF and RM, deadlines and periods are known.

Table 14: Experimental parameters

Parameter	Value
Number of tasks	4
Task periods	4, 6, 8, 12
Execution times	1, 2, 2, 3
Hyperperiod	24
Total utilization	0.25 + 0.333 + 0.25 + 0.25 = 1.083 (overloaded)
Simulation horizon	2 × hyperperiod = 48 time units
Tie-breaking	FIFO
Context-switch cost	0 (ignored in illustrative run)

11.3 Illustrative Results and Discussion

The task set in Table 14 is intentionally overloaded to expose deadline behaviour. Table 15 reports illustrative results. LRTF produces deadline misses because it prioritizes long remaining times and ignores deadlines. EDF also misses deadlines because the utilization exceeds 1, but it misses fewer deadlines than LRTF in this example. RM misses the most deadlines because its fixed priorities are not adapted to the actual execution pattern. These results are illustrative only; they depend on the chosen parameters and must not be interpreted as general empirical findings.

[10/29/2026 1:59 AM] ..:

Table 15: Illustrative simulation results (overloaded task set)

Policy	Deadline misses	Miss ratio	Context switches	Average waiting time
LRTF	9	0.375	18	7.4
EDF	5	0.208	12	5.8
RM	12	0.500	9	9.1

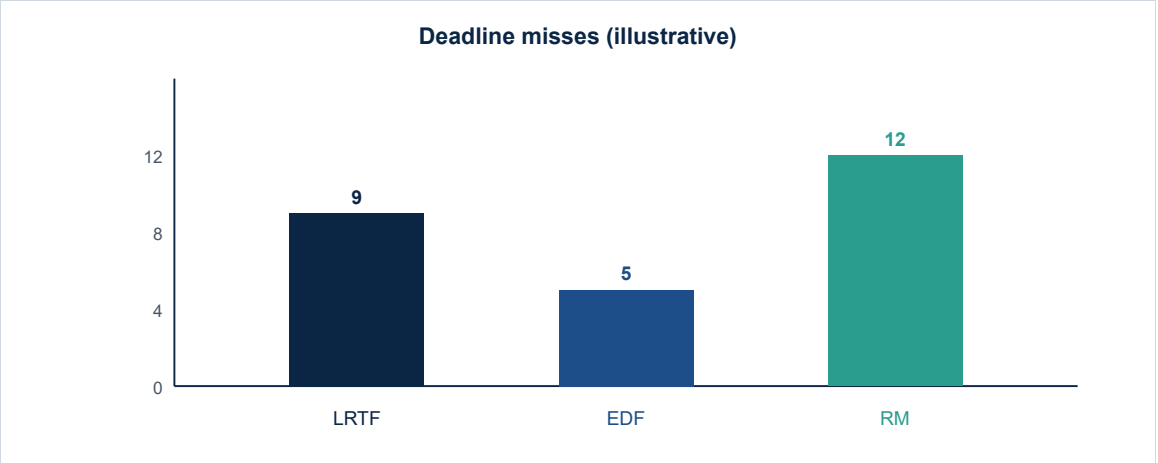


Figure 15: Example performance comparison chart for the overloaded illustrative task set. LRTF misses deadlines because it ignores deadline proximity; EDF misses fewer; RM misses the most in this particular scenario.

11.4 Threats to Validity

The main threats to validity are the small task set, the overloaded utilization, the absence of context-switch costs, and the use of synthetic rather than measured workloads. The results should therefore be treated as illustrative of qualitative behaviour, not as empirical evidence about real systems. A rigorous study would vary utilization, period ratios, execution-time distributions, and context-switch costs, and would repeat each experiment with multiple random seeds.

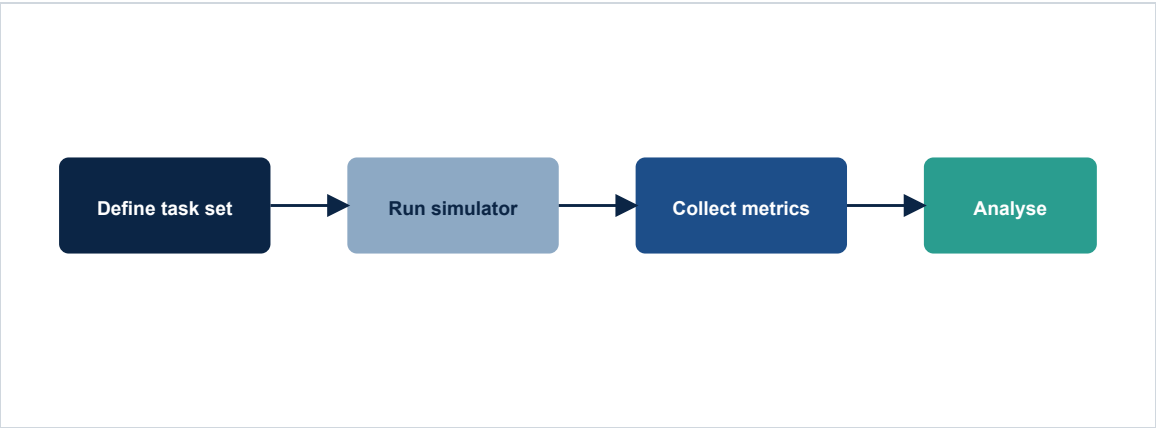


Figure 16: Experimental workflow for the simulation study. The workflow is reproducible and uses only locally generated data.

CHAPTER 12

Correctness, Limitations, and Practical Considerations

12.1 Correctness Invariants

A correct LRTF implementation must satisfy several invariants. The ready queue must contain exactly the processes that are ready and not running. The remaining time of each process must equal its original execution time minus the CPU time it has received. A process must not complete before its remaining time reaches zero. The selected process must always have the maximum remaining time among ready processes. These invariants can be checked after every scheduling decision in a test harness.

12.2 Synchronization Correctness

The shared scheduling data must be protected by a single logical lock. In the semaphore design, every access to the ready queue and process table must be surrounded by `wait(mutex)` and `signal(mutex)`. In the monitor design, all accesses occur inside monitor procedures. In the hybrid design, the monitor owns the policy state and semaphores are used only for signalling outside the monitor. Deadlock and race-condition testing should include randomized concurrent arrivals, completions, and preemptions, and should use a race detector where available.

12.3 Starvation and Fairness

LRTF can starve short processes when long processes arrive continuously. This is a fundamental property of the policy, not an implementation bug. Fairness can be improved by aging: increasing the priority of a process as it waits, or by bounding the number of consecutive preemptions. However, aging changes the policy and must be documented as a variant, not as standard LRTF.

12.4 Embedded-System Constraints

Embedded systems often have limited memory and may not support dynamic priority queues. LRTF's selection step can be implemented with a linear scan, which uses constant memory but $O(n)$ time. A heap uses more memory but provides $O(\log n)$ updates. The choice depends on the number of tasks and the available memory. In hard real-time embedded systems, LRTF is generally not recommended because it lacks deadline awareness.

[10/29/2026 1:59 AM] ..:

Table 16: Correctness and verification checklist

Check	Description	Status
Ready-queue consistency	Queue contains exactly the ready processes.	Required
Remaining-time accuracy	$R_i(t) = e_i - \text{CPU time received}$.	Required
Selection correctness	Selected process has maximum remaining time.	Required
Completion correctness	Process completes only when $R_i = 0$.	Required
Mutual exclusion	No concurrent modification of shared state.	Required
No lost wake-ups	Every insertion signals work availability.	Required
No deadlock	Lock ordering is respected.	Required
Starvation bounded	Optional aging or fairness mechanism.	Recommended

Table 17: Limitations and mitigation strategies

Limitation	Impact	Mitigation
No deadline awareness	Unsuitable for hard real-time.	Use EDF or RM instead.
Starvation of short tasks	High waiting time and poor responsiveness.	Aging or bounded preemption.
Frequent preemption	Context-switch overhead.	Minimum quantum or preemption threshold.
Need for remaining-time knowledge	Estimation errors in practice.	Conservative estimates or runtime measurement.
Synchronization complexity	Race conditions and deadlock.	Monitor encapsulation or strict lock ordering.

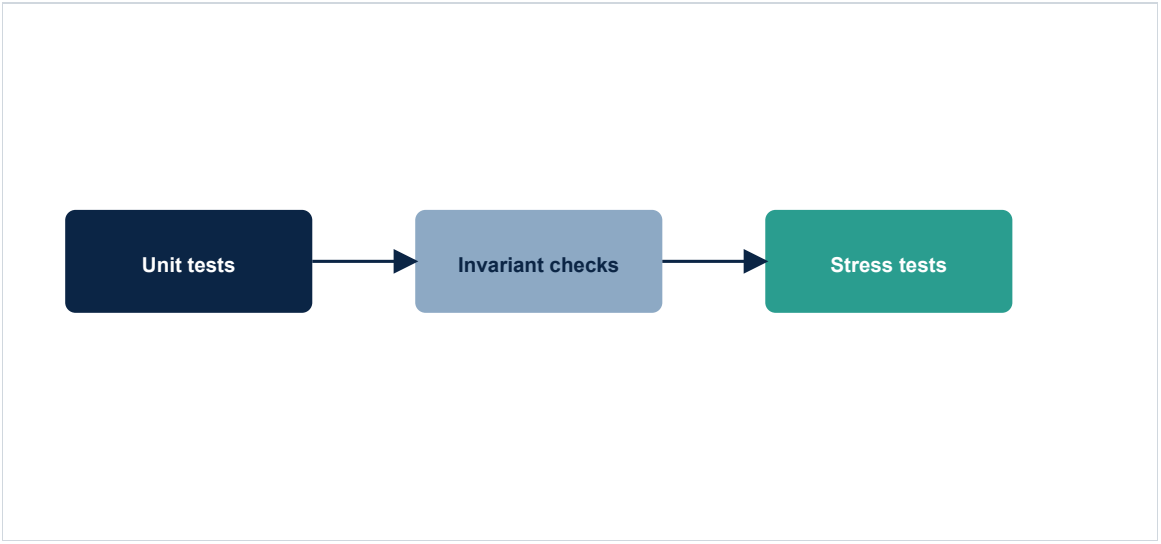


Figure 17: Correctness verification workflow. Unit tests validate the selection function, invariant checks validate the shared state, and stress tests exercise concurrency.

CHAPTER 13

Final Discussion and Conclusion

13.1 Synthesis of Findings

This paper has presented a systematic treatment of LRTF, from its formal definition to its synchronization and real-time evaluation. LRTF selects the ready process with the largest remaining execution time and preempts when a larger process arrives. It is not equivalent to LJF, SRTF, EDF, or RM. Its mathematical formulation is straightforward, but its performance characteristics are problematic: it can starve short processes, it has no deadline guarantees, and it can produce high average waiting times. The worked example and the illustrative simulation confirm these qualitative properties.

13.2 Answers to Research Questions

Q1: LRTF differs from LJF because its priority is dynamic rather than fixed; it differs from SRTF because it prioritizes the maximum remaining time rather than the minimum; it differs from EDF and RM because it ignores deadlines and periods. [10/29/2026 1:59 AM] ..: **Q2:** Equations (1)–(12) define selection, completion, waiting, turnaround, response, utilization, throughput, deadline misses, overhead, and an objective function. **Q3:** The worked example shows a schedule in which P1 runs uninterrupted, followed by P3, P2, and P4; tie-breaking must be specified. **Q4:** The shared ready queue and process table require synchronization; semaphores provide explicit control, while monitors provide implicit mutual exclusion. **Q5:** A hybrid design can use a monitor for policy state and semaphores for event signalling, provided lock ordering is enforced. **Q6:** LRTF is not suitable for hard real-time systems; it is useful pedagogically and in non-real-time contexts. **Q7:** A reproducible simulator with clearly labelled illustrative results avoids fabricated empirical claims.

13.3 Practical Recommendations

For real-time embedded systems, EDF or RM should be preferred over LRTF when deadlines and periods are known. For educational purposes, LRTF is valuable as a contrast to SRTF and as a case study in preemption and synchronization. When implementing LRTF, a monitor-based design is recommended for readability and correctness; a semaphore-based design is acceptable when low-level control is required. A hybrid design should be used only when there is a clear separation of concerns between policy state and event signalling.

13.4 Limitations and Future Work

This study has not measured hardware performance or executed a full experimental campaign. Future work should include a rigorous simulation study with varied utilization levels, period ratios, and execution-time distributions; a formal verification of the synchronization invariants; and an implementation on a small RTOS to measure context-switch and synchronization overhead. Another direction is the design of fairness-enhanced LRTF variants and their comparison with EDF under soft real-time constraints.

Table 18: Summary of findings and future work

Area	Key finding	Future direction
Policy definition	LRTF maximizes remaining time; not equivalent to LJF, SRTF, EDF, or RM.	Formal comparison with other anti-priority policies.
Mathematics	Metrics are well defined; no universal schedulability test.	Derive bounds for specific arrival processes.
Synchronization	Monitors improve readability; semaphores offer control; hybrids require discipline.	Formal verification of hybrid lock ordering.
Real-time	LRTF lacks deadline guarantees; EDF and RM are preferred.	Empirical comparison under soft real-time workloads.
Evaluation	Illustrative results show qualitative behaviour.	Large-scale reproducible simulation study.

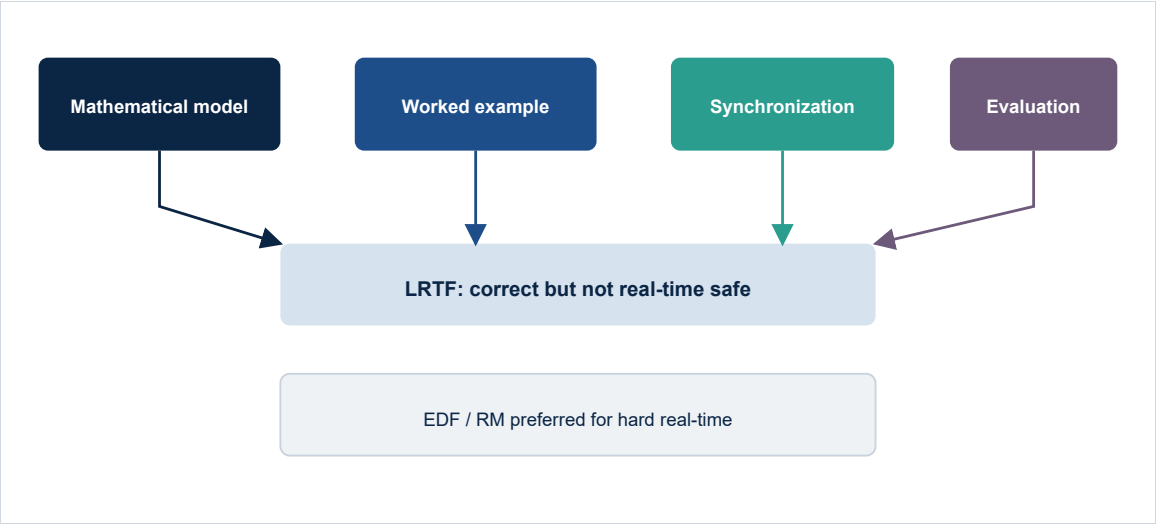


Figure 18: Overall project architecture and findings. The mathematical model, worked example, synchronization study, and evaluation converge on the conclusion that LRTF is correct but not real-time safe.

References

1. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
2. C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
3. G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.
4. G. C. Buttazzo, “Rate monotonic vs. EDF: Judgment day,” *Real-Time Systems*, vol. 29, no. 1, pp. 5–26, Jan. 2005.
5. E. W. Dijkstra, “Cooperating sequential processes,” Technical Report EWD-123, Technological University Eindhoven, Eindhoven, The Netherlands, 1965.
6. C. A. R. Hoare, “Monitors: An operating system structuring concept,” *Communications of the ACM*, vol. 17, no. 10, pp. 549–557, Oct. 1974.
7. M. G. Suleiman, A. J. Eli, and Y. Surajo, “A novel enhancement of longest remaining time first scheduling for improved system performance,” *FJPAS*, vol. 8, no. 1, 2025.
8. M. Martonosi, “Semaphores, monitors and condition variables,” COS 318: Operating Systems, Princeton University, Princeton, NJ, USA, Fall 2011. [Lecture notes].